

UNIT - I

SQL DATA STRUCTURE

Theory Notes

Schema

What You Will Learn

1. The basic structure for database software.
2. The basic SQL syntax.
3. How to create a new schema and confirm its contents are correct.

General Database Structure

The image in the source material describes what the architecture looks like in a typical database software ecosystem.

Level	Description
Database Software	It can be regarded as common database software, such as MySQL, PostgreSQL, MsSQL, etc. The source material uses MySQL because it is one of the most widely used database software systems.
Database	Different database structures can be separated according to different commercial meanings. For example, US stock data can be stored in one database and London stock data in another.
Schema	A collection of tables that can be split and grouped according to logic. Some table-detail metadata settings can be set at this layer, like a blueprint. In some database software, such as MySQL, Schema and Database are integrated into one.
Table	The key section of the database. Tables record different data entities, such as users and products. Each table is responsible for recording column size, type, default value, and related row data settings.
Data	The most valuable part of the database is the data itself. Different types of data can be stored.

Schema Syntax

Basic SQL Syntax

Usually in the database, when we want to create a Schema, the corresponding SQL syntax structure is as follows.

```
CREATE SCHEMA `new_schema` DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Here are a few basic points of SQL syntax:

- The all-capital part is the so-called keyword, which is the vocabulary that the SQL programming language needs to use. Because it has special meanings in SQL language, we cannot use it to name our schema, table, etc.
- A pair of ` symbols encloses new_schema. This is part of our own input, declaring that the name of the schema we want to create is new_schema.

Create Schema

The above statement can be divided into three parts to make it easier to understand:

4. The first component is the simplest; it creates a Schema and gives it a name.

```
CREATE SCHEMA `new_schema`
```

5. Set the character encoding because the symbols used for human language are very diverse. For example, Japanese and Arabic will have different symbol systems. The source uses the common 4-Byte UTF-8 Unicode Encoding series.
6. In the character encoding, there are derivatives. The example uses `utf8mb4_unicode_ci`, which is a derivative that can use emoji-related symbols. This is shown as an example in the source.

Most importantly, after the SQL statement is described, you need a semicolon (;) to indicate the endpoint of program execution.

After the statement has been executed and creation is complete, the source material shows that a new schema called "new_schema" can be confirmed.

Throughout the source material, example SQL schemas and queries are used for learning and practice. It encourages experimenting with the queries and writing queries independently.

`INFORMATION_SCHEMA.SCHEMATA` is the place where Schema data is stored in MySQL. The basic data of Schema can be viewed there.

Table

What You Will Learn

7. How to create, read, delete, and clean tables.
8. What a table does and what a table is responsible for.

Concept

A table is like the nanny of all data, and it is a cornerstone of the database. A good table design can significantly improve the maintainability and performance speed of the database.

The source introduces four characteristics of tables:

9. Tables record the metadata of each column, including the data type, default value, comments, etc.
10. Tables are responsible for managing the index, which can improve the speed of searching for data in the table.
11. Tables provide relationship definitions with other tables. For example, a user in the users table may have many order records in the orders table, and this affiliation needs to be managed in the table.
12. You can set the location where the data is actually stored on the computer hard drive.

SQL Syntax

Create Table

A practical table in SQL will look like this:

id	name	age
1	John	40
2	May	30
3	Tim	25

In order to create a table, the source starts with an empty table and uses a CREATE TABLE statement.

```
CREATE TABLE `new_schema`.`new_table` (  
  `id` INT NOT NULL COMMENT 'This is a primary index',  
  PRIMARY KEY (`id`)  
);
```

The statement can be split into three parts:

13. The first part establishes the table itself. The syntax changes from SCHEMA to TABLE. Because there may be many schemas in the database software, the schema and table are described together as new_schema.new_table.

```
CREATE TABLE `new_schema`.`new_table` ( ... other settings ... );
```

14. The second part defines the details of each field.

```
`id` INT NOT NULL COMMENT 'This is a primary index';
```

- id is the column name.
 - INT is the data type that will be stored in this column.
 - NOT NULL is a column attribute function.
 - The column has a COMMENT with the display text 'This is a primary index'.
15. The last part declares metadata of the table, such as setting the primary key of the table to id.

Check Table Definition

After creating the table, we can check whether the column settings are as expected, or whether there is a typo in a column name, by using `SHOW FULL COLUMNS`.

```
SHOW FULL COLUMNS FROM `new_schema`.`new_table`;
```

The result contains fields such as Field, Type, Collation, Null, Key, Default, Extra, Privileges, and Comment. In the example, `id` is an `INT` column, it is `NOT NULL`, it is a primary key, and its comment indicates that it is a primary index.

`SHOW` is a common statement for retrieving information from the database system level.

Destroy Table

The statement for deleting a table is simple:

```
DROP TABLE `new_schema`.`new_table`;
```

The `DROP` keyword means to remove a table from the database. It is dangerous because important data may be lost, so the source notes that it is rarely used in practical applications.

Clean Table

While a table is rarely deleted, testing may require the data state inside the table to be reset.

`TRUNCATE` deletes all data but keeps the table structure. The source notes that this statement should also be used with caution.

```
TRUNCATE TABLE `new_schema`.`new_table`;
```

Column Data Types

What You Will Learn

16. The different data types and when to use them.
17. The common column attribute functions provided by SQL.
18. The create and update column setting statements.

Each column is responsible for limiting the type and size of values that the data can store. Besides that, the column can enable some database functions as well.

Data Types

The source groups the data into three main types: number, datetime, and text. Database software can use these types and their extensions to set different data types. SQL also has special data types.

Number

19. `BIGINT`, `INT`, `MEDIUMINT`, `SMALLINT`, `TINYINT` represent integers. The farther to the left, the larger the number that can be stored. `INT` is common and can store values from -2147483648 to 2147483647.
20. `DOUBLE`, `FLOAT`, `DECIMAL` represent decimal values. `DECIMAL` can be set to `DECIMAL(5, 2)`, meaning the value has 5 digits with two digits after the decimal point, such as 666.88.

The source notes that `DOUBLE` and `FLOAT` are not precise. For high-accuracy data-system requirements, `DECIMAL` is recommended.

Datetime

- 21. DATE, MONTH, YEAR set date, month, and year.
- 22. DATETIME and TIMESTAMP are relatively complete date formats including year, month, day, hours, minutes, and seconds. DATETIME can accept a value such as 8888-01-01 00:00:00, while TIMESTAMP is limited to the range stated in the source material: 1970-01-01 00:00:01 to 2038-01-19 03:14:07.

Text

- 23. CHAR and VARCHAR store plain text. CHAR is suitable for fixed text length, such as currency abbreviations. VARCHAR is suitable for text whose length can change, and its length must be set.
- 24. TEXT and LONGTEXT can be used when the maximum length of text is unknown.

The source notes that a common MySQL VARCHAR setting is VARCHAR(45), meaning each item in the column can store 45 characters. A size that is too small can cause a column-size limit error.

Special Data Types

- 25. BINARY and BLOB store file-type data such as images or videos. BLOB (Binary Large Object) can also store large unstructured data. The source notes that it is rarely used in practice because managing files through database software can be difficult.
- 26. BOOLEAN stores logical values: true or false. The database replaces these with 1 or 0 respectively.
- 27. JSON is a common data format in modern data exchange.

Quick Data Type Reference

Category	Types	Purpose / Notes
Number	BIGINT, INT, MEDIUMINT, SMALLINT, TINYINT	Integer values
Number	DOUBLE, FLOAT, DECIMAL	Decimal values; DECIMAL is preferred when high precision is required
Datetime	DATE, MONTH, YEAR	Date-related values
Datetime	DATETIME, TIMESTAMP	Date and time including time components
Text	CHAR, VARCHAR	Plain text
Text	TEXT, LONGTEXT	Long or variable-length text
Special	BINARY, BLOB	Binary / large unstructured data
Special	BOOLEAN	True/false values
Special	JSON	Structured data exchange format

Column Attribute Functions

The demonstration statement in the source adds NOT NULL to the id column. Column attribute functions ensure that when new data is added to a table, the database processes the data format or content in advance.

These functions help users maintain data correctness. The source introduces the following common column attributes:

1. NOT NULL

NOT NULL means that the value in the field cannot be NULL. When NOT NULL is used, the column is forced to always contain a value.

In programming languages, NULL means nothing. Blank text is still a value, while NULL represents the absence of a value.

2. AUTO_INCREMENT

AUTO_INCREMENT automatically generates column values one by one using numbers. It is similar to a serial number. The database software helps avoid repeated serial numbers.

3. DEFAULT

DEFAULT sets a default value for a column because the data inserted into a field may be empty in some circumstances.

Database-System Differences

The source notes that different database systems may provide equivalent functionality using different syntax. For example, PostgreSQL can use SERIAL where MySQL uses AUTO_INCREMENT, and Oracle can use IDENTITY. The key is to understand what each function does rather than memorizing only its name.

Example of Column Attributes

```
CREATE TABLE example (  
  id INT NOT NULL AUTO_INCREMENT,  
  name VARCHAR(45) DEFAULT 'Unknown',  
  PRIMARY KEY (id)  
);
```

Create and Update Column

Columns are not static. When column definitions need to change, SQL provides ALTER TABLE and related commands.

Create / Add a Column

28. ALTER TABLE: When changing column settings, declare that the table is being altered.

29. ADD COLUMN: Add a new column and its settings. AFTER can be used to set the column order, which can improve readability when viewing the table column settings.

```
ALTER TABLE `new_schema`.`new_table`  
ADD COLUMN `name` VARCHAR(45) AFTER `id`;
```

Update / Change a Column

In SQL, updating a column is more like generating a new rule and overwriting the original version. The source uses CHANGE COLUMN and then supplies the new rules.

```
ALTER TABLE `new_schema`.`new_table`  
CHANGE COLUMN `id` `id` INT NOT NULL;
```

CHANGE COLUMN uses two consecutive names: the existing column name and the new column name, followed by the new rules. If the column name should not change, both names must be kept the same, such as id id.

Concept Summary

Command / Attribute	Main Function
CREATE SCHEMA	Creates a new schema
CREATE TABLE	Creates a new table
SHOW FULL COLUMNS	Displays table column definitions
DROP TABLE	Removes a table and its data
TRUNCATE TABLE	Removes all rows while keeping the table structure
NOT NULL	Prevents NULL values
AUTO_INCREMENT	Generates sequential numeric values
DEFAULT	Provides a default value
ALTER TABLE	Changes an existing table structure
ADD COLUMN	Adds a new column
CHANGE COLUMN	Changes column definition and can rename it

Movie Database Management and Analytical Reporting Case Study:

#Create a database and use the database

```
create database Movies_DB;

use Movies_DB;

create table movie
(
rel_year int,
mov_name varchar(100),
act_name varchar(50),
lang varchar(30),
dir_name varchar(100)
);
```

#Insert the data's

```
INSERT INTO MOVIES VALUES (1960,'Kalathur Kannamma','Kamal Haasan','Tamil','A. Bhimsingh');
INSERT INTO MOVIES VALUES (1962,'Kannum Karalum','Kamal Haasan','Malayalam','K. S. Sethumadhavan');
INSERT INTO MOVIES VALUES (1962,'Paadha Kaanikkai','Kamal Haasan','Tamil','K. Shankar');
INSERT INTO MOVIES VALUES (1962,'Paarthaal Pasi Theerum','Kamal Haasan','Tamil','A. Bhimsingh');
INSERT INTO MOVIES VALUES (1963,'Anandha Jodhi','Kamal Haasan','Tamil','V. N. Reddy');
INSERT INTO MOVIES VALUES (1963,'Vanambadi','Kamal Haasan','Tamil','G. R. Nathan');
INSERT INTO MOVIES VALUES (1970,'Maanavan','Kamal Haasan','Tamil','M. A. Thirumugam');
INSERT INTO MOVIES VALUES (1971,'Annai Velankanni','Kamal Haasan','Tamil','Thankappan');
INSERT INTO MOVIES VALUES (1971,'Anubhavangal Paalichakal','Mammootty','Malayalam','K. S. Sethumadhavan');
INSERT INTO MOVIES VALUES (1972,'Kurathi Magan','Kamal Haasan','Tamil','K. S. Gopalakrishnan');
INSERT INTO MOVIES VALUES (1973,'Arangetram','Kamal Haasan','Tamil','K. Balachander');
INSERT INTO MOVIES VALUES (1973,'Kaalachakram','Mammootty','Malayalam','K. Narayanan');
INSERT INTO MOVIES VALUES (1973,'Sollathaen Ninaikkiren','Kamal Haasan','Tamil','K. Balachander');
INSERT INTO MOVIES VALUES (1974,'Anbu Thangai','Kamal Haasan','Tamil','S. P. Muthuraman');
INSERT INTO MOVIES VALUES (1974,'Aval Oru Thodar Kathai','Kamal Haasan','Tamil','K. Balachander');
```

1. Retrieve all details from the movie table.

```
select * from MOVIES;
```

Solution:

rel_year	mov_name	act_name	lang	dir_name
1960	Kalathur Kannamma	Kamal Haasan	Tamil	A. Bhimsingh
1962	Kannum Karalum	Kamal Haasan	Malayalam	K. S. Sethumadhavan
1962	Paadha Kaanikkai	Kamal Haasan	Tamil	K. Shankar
1962	Paarthaal Pasi Theerum	Kamal Haasan	Tamil	A. Bhimsingh
1963	Anandha Jodhi	Kamal Haasan	Tamil	V. N. Reddy
1963	Vanambadi	Kamal Haasan	Tamil	G. R. Nathan
1970	Maanavan	Kamal Haasan	Tamil	M. A. Thirumugam
1971	Annai Velankanni	Kamal Haasan	Tamil	Thankappan
1971	Anubhavangal Paalichakal	Mammootty	Malayalam	K. S. Sethumadhavan
1972	Kurathi Magan	Kamal Haasan	Tamil	K. S. Gopalakrishnan
1973	Arangetram	Kamal Haasan	Tamil	K. Balachander
1973	Kaalachakram	Mammootty	Malayalam	K. Narayanan
1973	Sollathaen Ninaikkiren	Kamal Haasan	Tamil	K. Balachander
1974	Anbu Thangai	Kamal Haasan	Tamil	S. P. Muthuraman
1974	Aval Oru Thodar Kathai	Kamal Haasan	Tamil	K. Balachander

2. Display only movie names and actor names.

```
SELECT mov_name, act_name  
FROM movies;
```

Solution:

	mov_name	act_name
►	Kalathur Kannamma	Kamal Haasan
	Kannum Karalum	Kamal Haasan
	Paadha Kaanikkai	Kamal Haasan
	Paarthaal Pasi Theerum	Kamal Haasan
	Anandha Jodhi	Kamal Haasan
	Vanambadi	Kamal Haasan
	Maanavan	Kamal Haasan
	Annai Velankanni	Kamal Haasan
	Anubhavgal Paalichakal	Mammootty
	Kurathi Magan	Kamal Haasan
	Arangetram	Kamal Haasan
	Kaalachakram	Mammootty
	Sollathan Ninaikkiren	Kamal Haasan
	Anbu Thangai	Kamal Haasan
	Aval Oru Thodar Kathai	Kamal Haasan
	Gumasthavin Magal	Kamal Haasan
	Kanyakumari	Kamal Haasan

3. Display distinct movie languages.

```
select distinct lang from movies;
```

Solution:

Result Grid	
	lang
►	Tamil
	Malayalam
	Telugu
	Kannada
	Hindi
	Bengali
	Silent
	English

4. Retrieve unique actor names.

```
SELECT DISTINCT act_name  
FROM movies;
```

Solution:

act_name
Kamal Haasan
Mammootty
Rajinikanth
Mohanlal
Vijay
Silambarasan
Ajith Kumar
Suriya
Trisha
Nayanthara
Karthi
Dhanush
Dulquer Salm...
Amala Paul
Rashmika Ma...
Manju Warior
Sai Pallavi
Parvathy Thi...
Vijay Sethup...